Methodology

Methodology I

This section maps each step of the exploratory analysis to the tested function that implements it. Every function lives in `src/eda/`, takes a `pandas.DataFrame` in, and returns data out — no plotting, no file I/O — so the notebook, the analysis script, and this manuscript all reach the same code.

Dataset loading and schema I

`src/eda/dataset.py::load_dataset()` reads the shipped CSV and coerces the numeric columns with `pandas.to_numeric(errors="coerce")`. Coercion is deliberate: a blank or non-numeric cell becomes `NaN` rather than raising or being silently dropped, so missingness is *surfaced* and handled in an explicit later step. The column roles are declared once in a frozen `DatasetSchema` (an identifier column, a categorical group column, and three numeric features), which downstream functions consult instead of re-sniffing dtypes.

Cleaning: explicit, reported row removal I

`src/eda/cleaning.py::clean_dataset()` drops any row missing a numeric feature and returns a `CleaningReport` recording how many rows entered, how many remained, and how many were dropped. This exemplar makes a deliberate choice — listwise deletion with a visible count — rather than imputation, because a first EDA pass should *see* the missingness, not paper over it. A separate `normalize_numeric()` z-score standardizes the numeric columns (mean 0, sample standard deviation 1), with a guard that maps a constant or single-row column to zeros instead of producing `inf`/`NaN`.

Descriptive statistics I

`src/eda/statistics.py::summary_statistics()` returns one `ColumnSummary` per numeric column — count of non-missing observations, mean, sample standard deviation, minimum, median, and maximum — computed with real pandas aggregation. `group_means()` returns the mean of each numeric feature grouped by the categorical column, sorted by group name for deterministic output.

Correlation structure I

`src/eda/correlation.py::correlation_matrix()` returns the Pearson correlation matrix of the numeric columns [@tukey1977eda]. The companion `strongest_pairs(matrix, top_n)` ranks the distinct off-diagonal feature pairs by absolute correlation while preserving sign — usually the single most useful artifact of a first pass, because it points directly at the relationships worth investigating. Each unordered pair appears exactly once.

Figure-data preparers I

Plotting is kept out of the library entirely. `src/eda/figures.py` returns *plot-ready data structures* — bin counts and edges for a histogram, a square value grid plus labels for a correlation heatmap, and sorted category counts for a bar chart — as frozen dataclasses of plain numbers. The thin analysis script (`scripts/eda_analysis.py`) and notebook cells consume these structures and call matplotlib. This keeps the library importable on a headless machine and makes every preparer testable without a display backend.

Zero-mock testing methodology I

The project is governed by a strict zero-mock policy, evaluated by running `uv run pytest projects/templates/template_eda_notebook/tests` during the build.

1. **Library tests** exercise every public function against the shipped CSV or a tiny hand-built frame with values chosen so the expected statistic is exact (e.g. `weight = 2 * height` gives a correlation of exactly `+1.0`). No `unittest.mock`, no `MagicMock`, no `@patch`.
2. **Script test** runs `run_eda()` against a temporary output root and asserts that real PNG figures and a real summary CSV are written.
3. **Notebook test** parses the real `.ipynb` and asserts it is valid nbformat, that every name imported `from src` exists in the library's public surface, and that no cell defines its own `def`/`class`.

Zero-mock testing methodology II

4. **Coverage gate**: CI enforces a 90% statement-coverage gate on `projects/templates/template_eda_notebook/src/`; the live figure is tracked in `docs/_generated/COUNTS.md`.

Figure generation contract I

Each figure in `03_results.md` maps to a figure-data preparer in `src/eda/figures.py`: `histogram_data` → height histogram, `correlation_heatmap_data` → feature-correlation heatmap, and `group_count_data` → per-group row counts. Captions name the preparer and the key parameters (bin count, value range) so reviewers can navigate from the PDF to the code without inferring hidden defaults.